



铁头山羊STM32教程

[SPI]

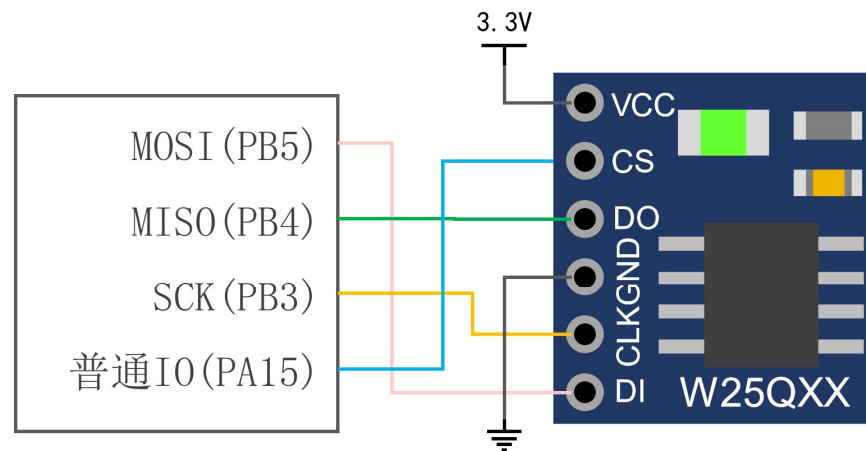
5.7. W25Q64实验(上)

版权归东莞市明玉科技有限公司所有

未经允许不得传播

表49 SPI1重映像

复用功能	SPI1_REMAP = 0	SPI1_REMAP = 1
SPI1_NSS	PA4	PA15
SPI1_SCK	PA5	PB3
SPI1_MISO	PA6	PB4
SPI1_MOSI	PA7	PB5

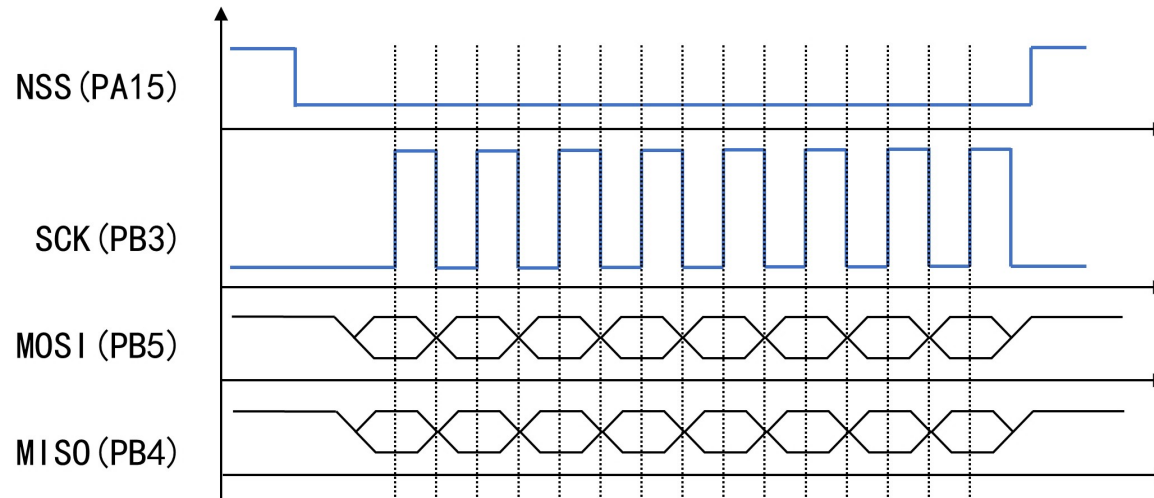


版权归东莞市明玉科技有限公司所有
未经允许不得传播

表35 调试端口映像

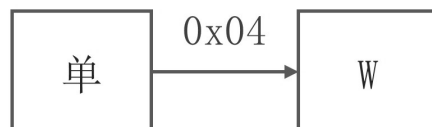
SWJ_CFG [2:0]	可能的调试端口	SWJ I/O引脚分配				
		PA13/ JTMS/ SWDIO	PA14/ JTCK/ SWCLK	PA15/ JTDI	PB3/ JTDO/ TRACESWO	PB4/ NJTRST
000	完全SWJ(JTAG-DP + SW-DP) (复位状态)	I/O不可用	I/O不可用	I/O不可用	I/O不可用	I/O不可用
001	完全SWJ(JTAG-DP + SW-DP) 但没有JNTRST	I/O不可用	I/O不可用	I/O不可用	I/O不可用	I/O可用
010	关闭JTAG-DP, 启用SW-DP	I/O不可用	I/O不可用	I/O可用	I/O可用 ⁽¹⁾	I/O可用
100	关闭JTAG-DP, 关闭SW-DP	I/O可用	I/O可用	I/O可用	I/O可用	I/O可用
其它	禁用					

1. I/O 口只可在不使用异步跟踪时使用。



```
GPIO_WriteBit(GPIOA, GPIO_Pin_15, Bit_SET);
```

版权归东莞市明玉科技有限公司所有
未经允许不得传播



11.2.5 Write Disable (04h)

The Write Disable instruction (Figure 5) resets the Write Enable Latch (WEL) bit in the Status Register to a 0. The Write Disable instruction is entered by driving /CS low, shifting the instruction code "04h" into the DI pin and then driving /CS high. Note that the WEL bit is automatically reset after Power-up and upon completion of the Write Status Register, Page Program, Sector Erase, Block Erase and Chip Erase instructions.

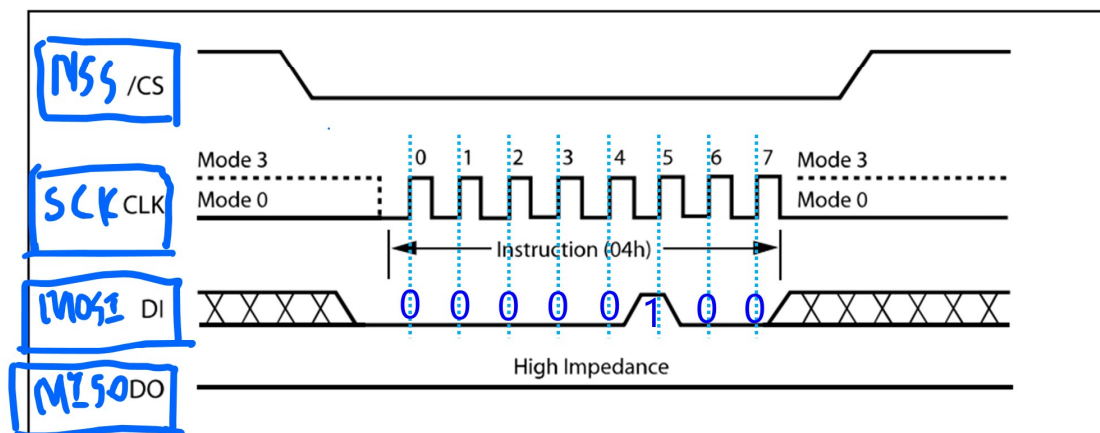


Figure 5. Write Disable Instruction Sequence Diagram

数据宽度

8b / 16b

极性

高 / 低

相位

1 / 2

比特位传输顺序

MSB / LSB

版权归东莞市明玉科技有限公司所有

未经允许不得传播

- 1. W25Q64的存储结构
- 2. 使用W25Q64存储数据
- 3. 使用W25Q64读取数据



```
void App_W25Q64_SaveByte(uint8_t Byte); // 使用W25Q64保存一个字节  
uint8_t App_W25Q64_LoadByte(void);      // 把保存的字节读出来
```

版权归东莞市明玉科技有限公司所有
未经允许不得传播

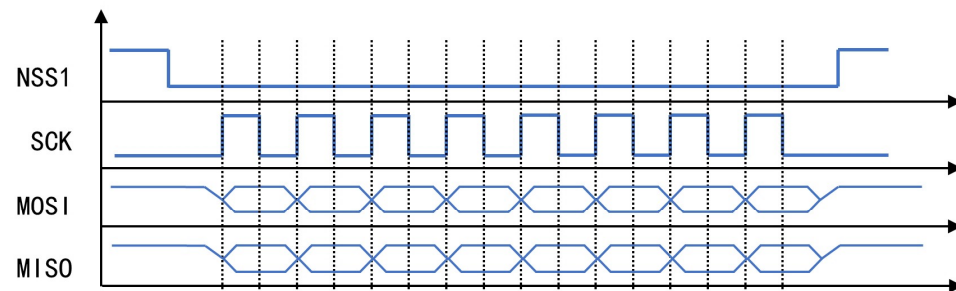
```
void App_SPI_MasterTransmitReceive(SPITypeDef *SPISx,    // SPI的名称
                                   const uint8_t *pDataTx, // 要发送的数据
                                   uint8_t *pDataRx,  // 接收到的数据
                                   uint16_t Size);      // 收发数据的数量
```

作用：使用SPI总线收发数据

```
// 接收5个字节，同时发送5个字节
uint8_t buffer[] = {0x01, 0x02, 0x03, 0x04, 0x05};
App_SPI_Master_TransmitReceive(...,buffer, buffer, 5);

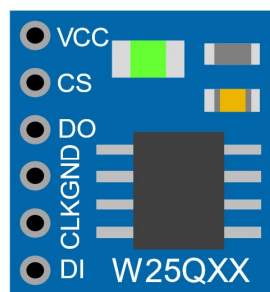
// 只发送5个字节，同上

// 只接收5个字节
uint8_t buffer[] = {0xff, 0xff, 0xff, 0xff, 0xff, 0xff};
App_SPI_Master_TransmitReceive(...,buffer, buffer, 5);
```

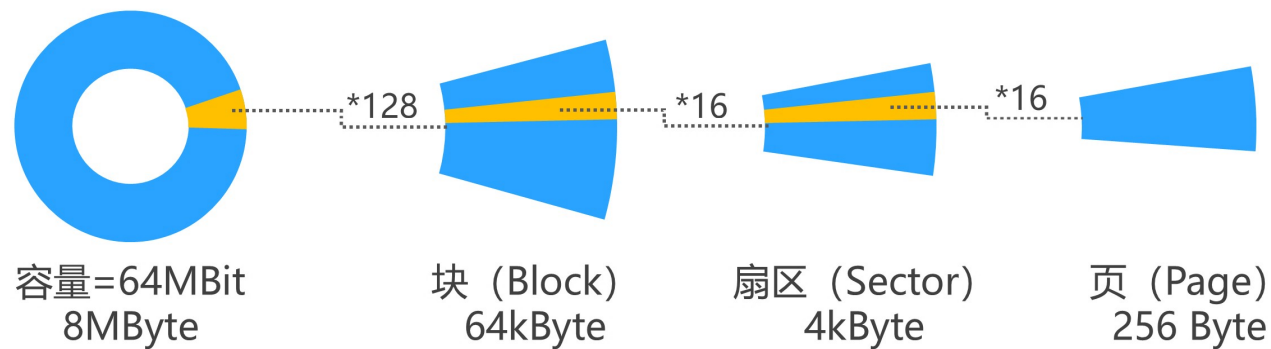


版权归东莞市明玉科技有限公司所有

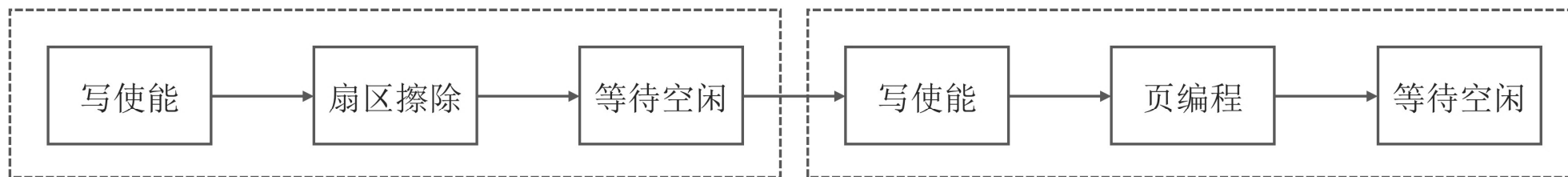
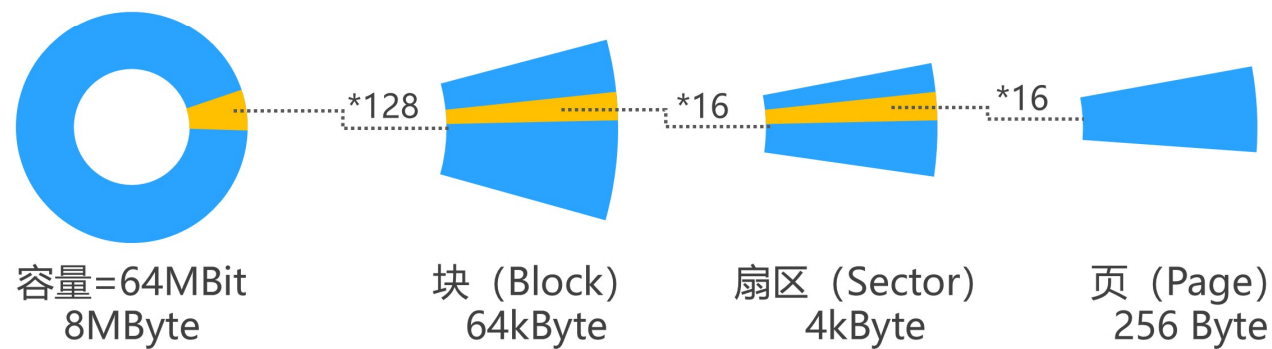
未经允许不得传播



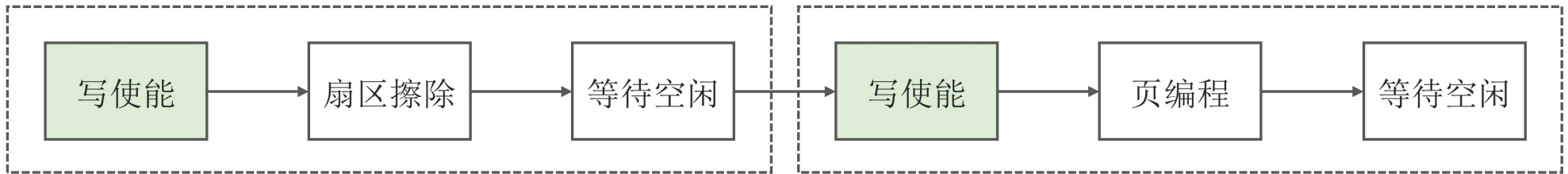
W25Q64



版权归东莞市明玉科技有限公司所有
未经允许不得传播



版权归东莞市明玉科技有限公司所有
未经允许不得传播



11.2.4 Write Enable (06h) 主机发0x06

The Write Enable instruction (Figure 4) sets the Write Enable Latch (WEL) bit in the Status Register to a 1. The WEL bit must be set prior to every Page Program, Sector Erase, Block Erase, Chip Erase and Write Status Register instruction. The Write Enable instruction is entered by driving /CS low, shifting the instruction code "06h" into the Data Input (DI) pin on the rising edge of CLK, and then driving /CS high.

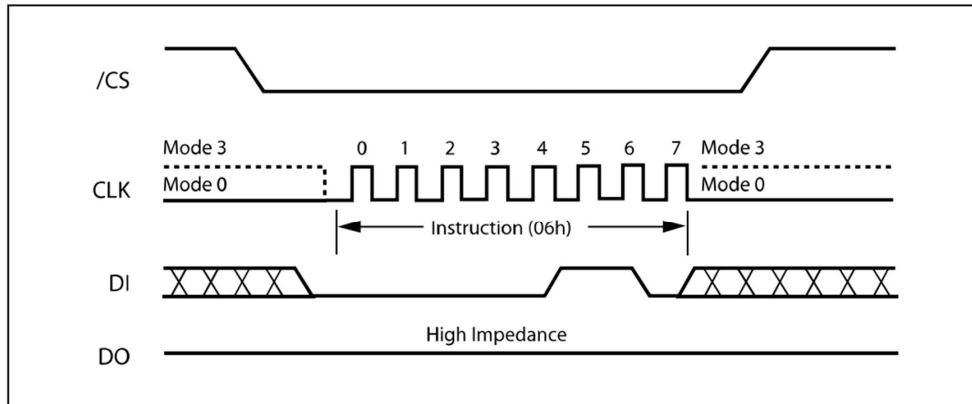


Figure 4. Write Enable Instruction Sequence Diagram

```
uint8_t buffer[10]; // 声明一个数组
```

```
// #1. 写使能
```

```
buffer[0] = 0x06;
```

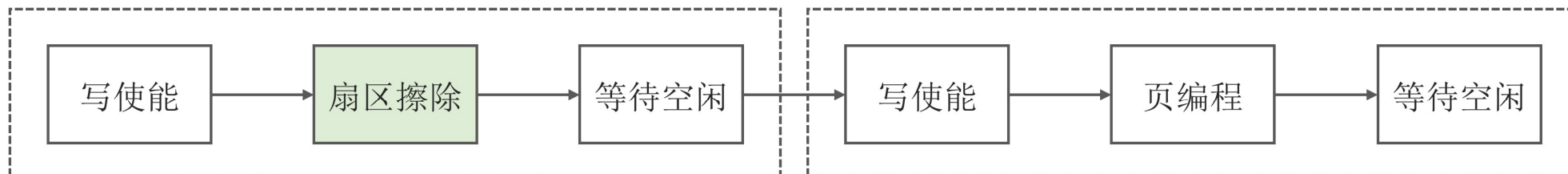
```
GPIO_WriteBit(GPIOA, GPIO_Pin_12, Bit_RESET); // NSS=0
```

```
App_SPI_MasterTransmitReceive(SPI1, buffer, buffer, 1);
```

```
GPIO_WriteBit(GPIOA, GPIO_Pin_12, Bit_SET); // NSS=1
```

版权归东莞市明玉科技有限公司所有

未经允许不得传播



11.2.17 Sector Erase (20h) 主机发0x20 + 24位地址

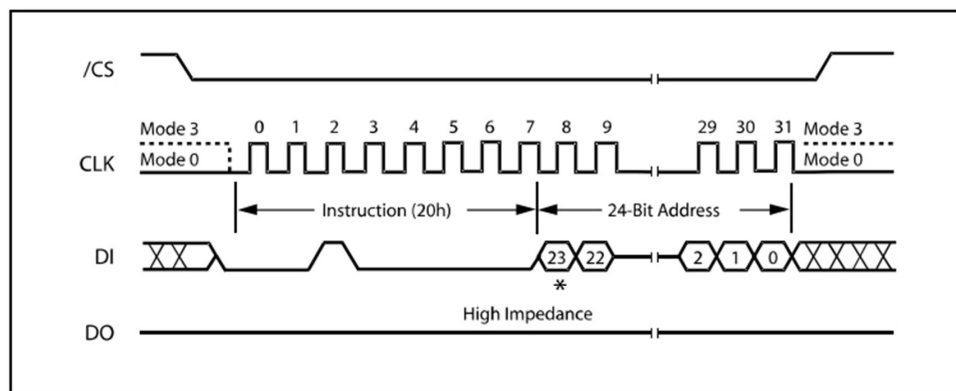


Figure 17. Sector Erase Instruction Sequence Diagram

// #2. 扇区擦除

```

buffer[0] = 0x20;
buffer[1] = 0x00;
buffer[2] = 0x00;
buffer[3] = 0x00;
  
```

```

GPIO_WriteBit(GPIOA, GPIO_Pin_12, Bit_RESET); // NSS=0
  
```

```

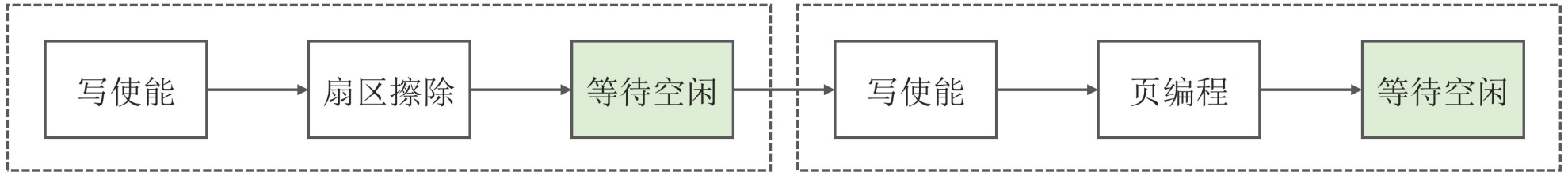
App_SPI_MasterTransmitReceive(SPI1, buffer, buffer, 4);
  
```

```

GPIO_WriteBit(GPIOA, GPIO_Pin_12, Bit_SET); // NSS=1
  
```

版权归东莞市明玉科技有限公司所有

未经允许不得传播



状态
寄存器1

S7	S6	S5	S4	S3	S2	S1	S0
SRP0	SEC	TB	BP2	BP1	BP0	WEL	BUSY

11.2.6 Read Status Register-1 (05h) and Read Status Register-2 (35h)

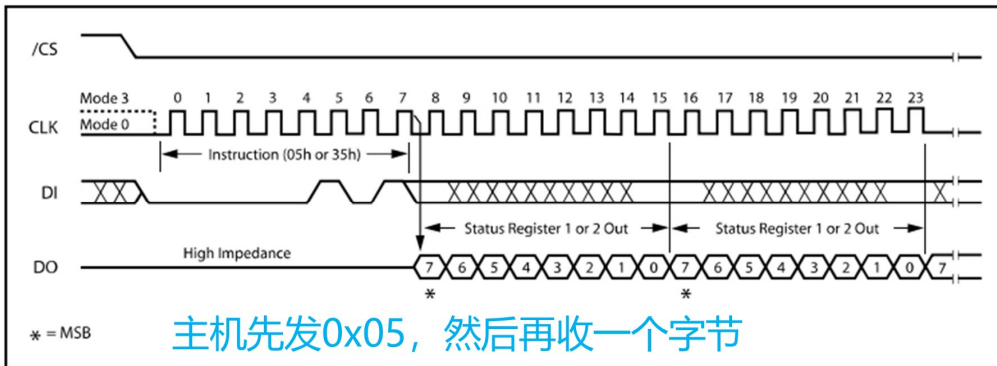


Figure 6. Read Status Register Instruction Sequence Diagram

// #3. 等待空闲

while(1){

`GPIO_WriteBit(GPIOA, GPIO_Pin_12, Bit_RESET); // NSS=0`

`// 写0x05`

`buffer[0] = 0x05;`

`App_SPI_MasterTransmitReceive(SPI1, buffer, buffer, 1);`

`// 读一个字节`

`buffer[0] = 0xff`

`App_SPI_MasterTransmitReceive(SPI1, buffer, buffer, 1);`

`GPIO_WriteBit(GPIOA, GPIO_Pin_12, Bit_SET); // NSS=1`

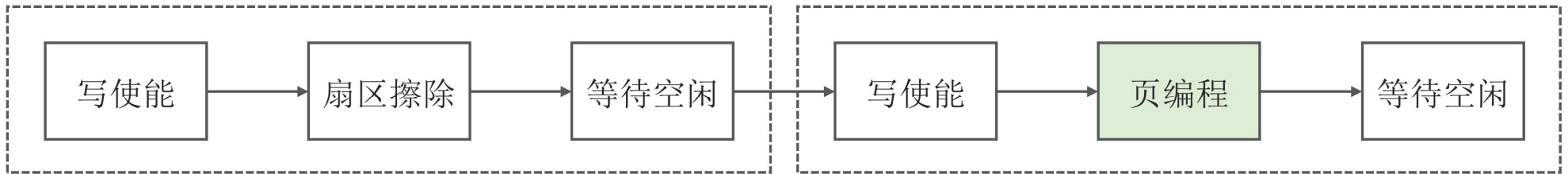
`// 如果BUSY=0则退出等待`

`if((buffer[0] & 0x01) == 0) break;`

}

版权归东莞市明玉科技有限公司所有

未经允许不得传播



11.2.15 Page Program (02h) 主机发0x02 + 24位地址 + 要写的数

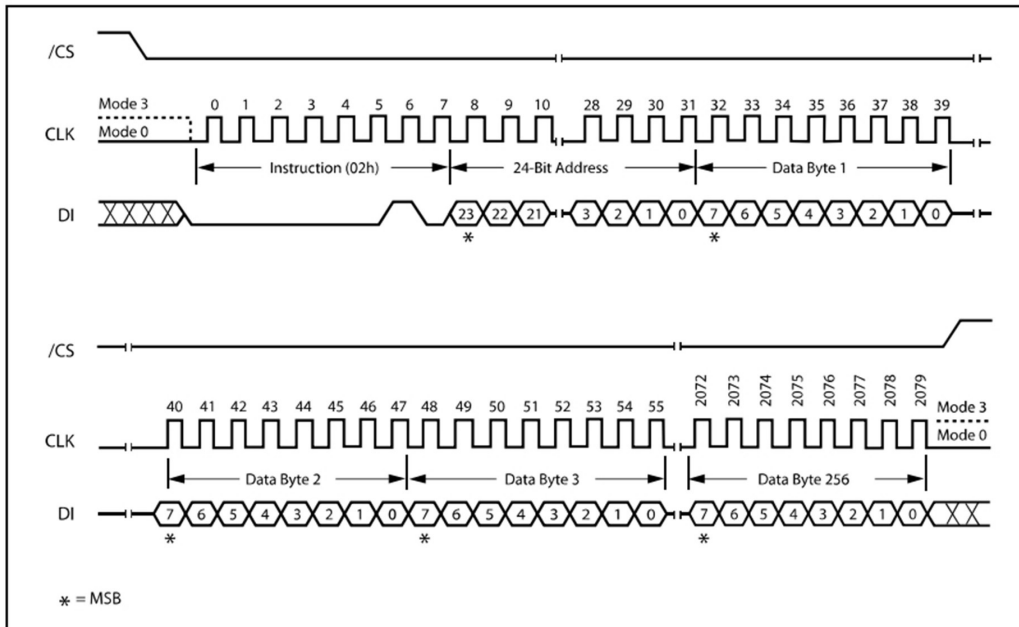


Figure 15. Page Program Instruction Sequence Diagram

// #5. 页编程

```

buffer[0] = 0x02;
buffer[1] = 0x00;
buffer[2] = 0x00;
buffer[3] = 0x00;
buffer[4] = Byte;
  
```

```
GPIO_WriteBit(GPIOA, GPIO_Pin_12, Bit_RESET); // NSS=0
```

```
App_SPI_MasterTransmitReceive(SPI1, buffer, buffer, 5);
```

```
GPIO_WriteBit(GPIOA, GPIO_Pin_12, Bit_SET); // NSS=1
```

版权归东莞市明玉科技有限公司所有

未经允许不得传播

11.2.8 Read Data (03h) 主机发0x03 + 24位地址，然后读取数据

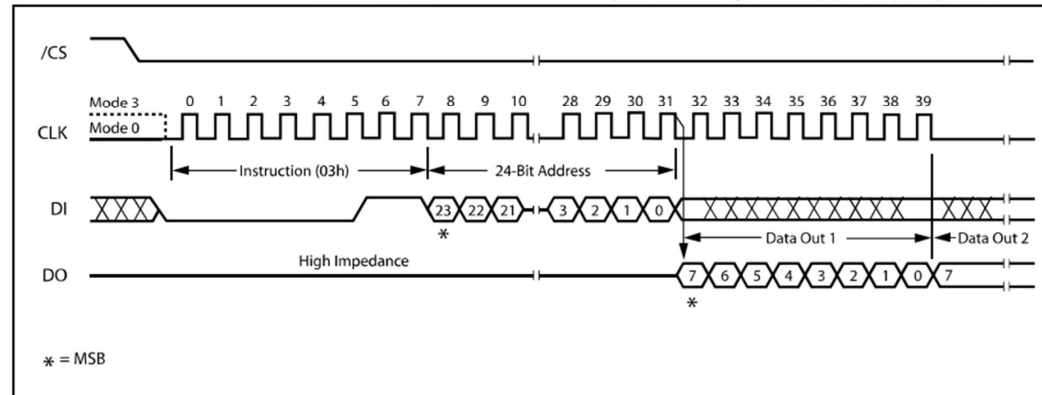


Figure 8. Read Data Instruction Sequence Diagram

// 读取一个字节

```
buffer[0] = 0x03; buffer[1] = 0x00; buffer[2] = 0x00; buffer[3] = 0x00;
```

```
GPIO_WriteBit(GPIOA, GPIO_Pin_12, Bit_RESET); // NSS=0
```

```
App_SPI_MasterTransmitReceive(SPI1, buffer, buffer, 4); // 发0x03+24位地址
```

```
buffer[0] = 0xff;
```

```
App_SPI_MasterTransmitReceive(SPI1, buffer, buffer, 1); // 收一个字节
```

```
GPIO_WriteBit(GPIOA, GPIO_Pin_12, Bit_SET); // NSS=1
```

```
return buffer[0];
```

版权归东莞市明玉科技有限公司所有

非经允许不得传播



```
void App_W25Q64_SaveByte(uint8_t Byte); // 使用W25Q64保存一个字节  
uint8_t App_W25Q64_LoadByte(void); // 把保存的字节读出来
```

```
uint8_t a = 0;  
  
int main(void){  
    App_SPI1_Init();  
    App_W25Q64_SaveByte(0x12); // 将字节0x12保存在W25Q64的0x000000  
    a = App_W25Q64_LoadByte(); // 将保存的字节读出来  
  
    while(1){}  
}
```

版权归东莞市明玉科技有限公司所有
未经允许不得传播